DYNAMIC CONTEXT-AWARE TRIES FOR KNOWLEDGE GRAPH-CONSTRAINED LARGE LANGUAGE MODEL GENERATION

BERNARD KIRK ADJANOR KATAMANSO
ERICA AMONOR
JOSEPH OSEI NYARKO
JESSICA AFUA ETORNAM NSAFOAH

Supervised By: DR ERIC AFFUM

University of Mines and Technology
Faculty of Computing and Mathematical Science
Computer Science and Engineering Department

July ,2026

Presentation Outline

1. Introduction
2. Statement of Problem
3. Aims and Objectives
4. Methodology
5. Results and Evaluation
6. Discussion and Contribution
7. Recommendation
8. References

Introduction

- LLMs are now central to AI applications, but they often hallucinate and produce factually incorrect outputs (Brown *et al.*, 2020)
- KGQA demands answers that follow real graph structure (He *et al.*, 2021; Lan *et al.*, 2022); RAG and constrained decoding both try to anchor generation to verified facts (Lewis *et al.*, 2020; De Cao *et al.*, 2021; Willard and Louf, 2023; Geng *et al.*, 2023)
- Existing constrained decoding (GCR, DoG) guarantees validity, not relevance, leaving room for technically correct but irrelevant paths (Geng *et al.*, 2023; Li *et al.*, 2025)

Statement of Problem

- GCR's trie is built before decoding starts, so it can't adapt to the question's intent or the evolving reasoning chain (Luo *et al.*, 2025)
- This forces the LLM back onto internal knowledge to filter out structurally valid but semantically irrelevant paths (Li *et al.*, 2025; Luo *et al.*, 2025)

Project Objectives(1/3)

- Characterize the constraint permissiveness problem by measuring the Semantic Irrelevance Ratio (SIR) of GCR's static KG-Tries on WebQSP, stratified by hop depth
- Design a symbolic constraint oracle (TypeOracle) using a range gate and a type gate to filter candidate KG paths, avoiding the cost and threshold sensitivity of embedding-based approaches

Project Objectives(2/3)

- Implement DCA-Trie v1, a static filtering variant applied at trie-construction time, keeping the false negative rate on gold paths below 5% on the WebQSP validation set
- Implement DCA-Trie v2, a dynamic variant that updates the trie after each committed triplet, conditioned on the question and the partial reasoning chain

Project Objectives(3/3)

- Evaluate DCA-Trie v1 and v2 against the GCR baseline and an unconstrained chain-of-thought baseline on WebQSP, measuring Hits@1, F1, structural faithfulness, SIR, and average trie size

Tools and Facilities Used (1/2)

Software Tools:

- Python
- HuggingFace Transformers
- GCR Framework
- Sentence-transformers (all-MiniLM-L6-v2)
- PyTorch
- NetworkX
- Freebase (WebQSP and CWQ Datasets)
- NVIDIA GeForce RTX 4090 (24 GB)

Tools and Facilities Used (2/2)

Facilities:

- NVIDIA GeForce RTX 4090 GPU (24 GB VRAM)
- Online academic databases (ACM Digital Library, arXiv, Google Scholar, ACL Anthology)

Methods Used

- Literature Review and Theoretical Baseline
- Algorithm Design
- Type Oracle & Algorithm Implementation
- Experimental Benchmarking and Evaluation

Methodology

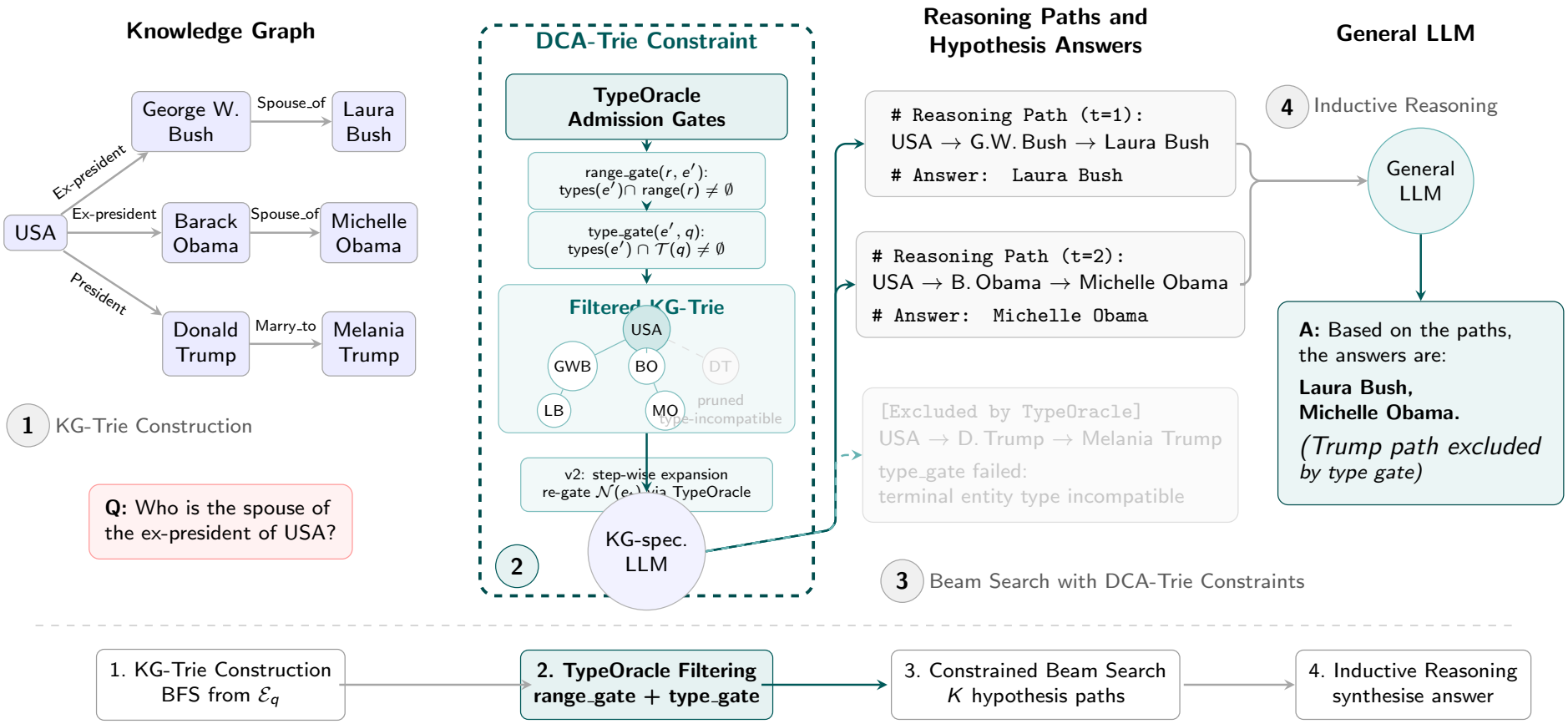

Figure: DCA-Trie System Architecture.

Upgraded Constraint Specification

- **Baseline Formalization (GCR/DoG):**

$$W_{\text{val}}^{\text{GCR}}(t) = f(G, E_q)$$

- **Proposed Formalization (DCA-Trie):**

$$W_{\text{val}}^{\text{DCA}}(t) = f(G, E_q, q, y_{<t})$$

Design Conditions

- **Condition F (Structural Faithfulness):** 100% structural grounding in KG triplets[cite: 1].
- **Condition R (Semantic Relevance Reduction):** Lower Semantic Irrelevance Ratio ($\overline{\text{SIR}}$)
- **Condition P (Recall Preservation):** False Negative Rate (FNR) on gold paths $< 5\%$

Diagnostic Metric: SIR Definition

SIR measures the fraction of structurally valid paths permitted by the constraint oracle that are semantically incompatible with the input question.

Step-Level Calculation:

$$\mathrm{SIR}(q,t) = \frac{\sum_{p \in \mathcal{P}(q,t)} \mathrm{irrelevant}(p,q)}{|\mathcal{P}(q,t)|}$$

Diagnostic Metric: Corpus-Level SIR

Corpus-Level Average:

$$\overline{\text{SIR}} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{L_q} \sum_{t=1}^{L_q} \text{SIR}(q, t)$$

- Evaluates overall constraint search space permissiveness across the benchmark dataset.
- Provides a quantitative diagnostic metric independent of downstream model generation accuracy.

Implementation Variants: DCA-Trie v1

Static Symbolic Filtering

- Filters BFS-enumerated paths through the Type Oracle prior to building the static prefix tree (`marisa_trie`).
- Preserves static trie lookup efficiency ($O(d)$ time complexity) while pruning irrelevant branches offline.

Implementation Variants: DCA-Trie v2

Step-Wise Symbolic Expansion

- Dynamically expands the trie step-by-step only when an entity token e_t is committed to the reasoning path.
- Applies local property range and answer type gate checks to candidate neighbors $\mathcal{N}(e_t)$ during token generation.

Results

- **Sensor Performance**
- **Data Transmission**
- **Fault Detection**
- **System Reliability**
- **Cost-Effectiveness**

Results: Comparative Evaluation

Metric / Method	Baseline (GCR)	DCA-Trie v1 (Static)	DCA-Trie v2 (Dynamic)
Hits@1	**91.6%**	86.4%	54.0%
F1 Score	**72.1%**	61.6%	—
Structural Faithfulness	**100.0%**	**100.0%**	**100.0%**
Path Reduction	0.0% (4.10M)	**14.5% (3.51M)**	—
Corpus SIR	1.000	**0.145**	—
Gold Path FNR	0.0%	**3.3% (Type) / 2.9% (Range)**	—
Latency / Overhead	6.35s / q	**6.38s / q (+0.5%)**	∼8,000s (Partial)

Evaluated on WebQSP (1,628 test queries) using Llama-3.1-8B with group-beam search ($k = 10$).

Discussion

- IoT sensors enabled **real-time monitoring** of motor health, showing feasibility of low-cost predictive maintenance solutions.
- Machine learning models (Random Forest, KNN) achieved **high accuracy in fault classification**, outperforming traditional threshold-based methods.
- **ThingSpeak cloud platform** ensured reliable data storage and visualization, confirming open-source IoT tools are effective for predictive maintenance.
- The system is **cost-effective** and scalable for industries, though future work should include more sensors.

Demonstration

Demonstration of project

Conclusion

- The IoT-based predictive maintenance system successfully monitored electric motor conditions in real-time.
- Machine learning models (Random Forest, KNN) effectively classified motor faults with high accuracy.
- The system proved cost-effective and reliable compared to traditional maintenance methods.

Recommendations

- Expand system with additional sensors (vibration, voltage, power factor) for enhanced fault detection.
- Train ML models with larger, more diverse datasets to improve generalization.
- Integrate edge computing to reduce cloud dependence and latency.
- Collaborate with industries for real-world deployment, testing, and scalability.

References

- Him, S., Kim, Y., and Lee, S. (2019), "IoT-based predictive maintenance for industrial equipment", *IEEE Access*, Vol. 7, pp. 113467–113477.
- Martins, R., Silva, A., and Pereira, J. (2020), "Condition monitoring of induction motors based on IoT and machine learning", *Journal of Intelligent Manufacturing*, Vol. 31, No. 5, pp. 1119–1132.

References

- Nangia, V., Gopal, A., and Mishra, R. (2020), "Predictive maintenance framework for electric motors using machine learning", *International Journal of Prognostics and Health Management*, Vol. 11, No. 3, pp. 1–12.

Thank You!

Thank You!!!